OOP S24 MODEL ANSWER MSBTE

This Study Material is provided by Campusifyplus.in ©

Made by Team Campusify

THIS IS THE FREE STUDY MATERIAL BY CAMPUSIFY!!

Be Part of Our Community

Telegram Channel - https://t.me/Campusifyy

Whatsapp Channel - https://tinyurl.com/Campusify-channel

Campusify is building the Biggest Diploma/Engineering Student Community in Maharashtra. Be part of the movement!

Join Channel for MSBTE Study Materials



Earn by Referring Campusify Course

Presented by **Campusify** | **10K** Active Members | **2.7L** Students Helped within **1 year**

Refer Our course with your **Third year** friend or cousin and get **10%** commission on each referral.

For More Details : WhatsApp Us at **+91 8291934640**

TY Course Link - https://www.campusifyplus.site

Join Channel for MSBTE Study Materials

SUMMER-2024 EXAMINATION
Model Answer – Only for the Use of RAC Assessors

Subject Name: Object Oriented Programming using C++ **Subject Code: 22316**

Important Instructions to examiners:

1) The answers should be examined by key words and not as word-to-word as given in the model answer scheme.
2) The model answer and the answer written by candidate may vary but the examiner may try to assess the understanding level of the candidate.
3) The language errors such as grammatical, spelling errors should not be given more Importance (Not applicable for subject English and Communication Skills.
4) While assessing figures, examiner may give credit for principal components indicated in the figure. The figures drawn by candidate and model answer may vary. The examiner may give credit for any equivalent figure drawn.
5) Credits may be given step wise for numerical problems. In some cases, the assumed constant values may vary and there may be some difference in the candidate's answers and model answer.
6) In case of some questions credit may be given by judgement on part of examiner of relevant answer based on candidate's understanding.
7) For programming language papers, credit may be given to any other program based on equivalent concept.
8) As per the policy decision of Maharashtra State Government, teaching in English/Marathi and Bilingual (English + Marathi) medium is introduced at first year of AICTE diploma Programme from academic year 2021-2022. Hence if the students write answers in Marathi or bilingual language (English +Marathi), the Examiner shall consider the same and assess the answer based on matching of concepts with model answer.

Q. No.	Sub Q. N.	Answer	Marking Scheme
1.		**Attempt any FIVE of the following:**	**10M**
	a)	**State any two features of object oriented programming.**	**2M**
	Ans.	**Features of object oriented programming are:** 1. **Objects:** Objects are the fundamental building blocks of OOP. They represent real-world entities with attributes (data) and methods (functions) that define their behavior. 2. **Classes:** A class that defines the properties and methods of an object. A class that creates multiple objects of the same kind. 3. **Encapsulation:** Encapsulation is the process of bundling data and methods together into a single unit, protecting the data from direct access outside the object. This ensures data integrity and promotes modularity. 4. **Inheritance:** Inheritance allows new classes to inherit properties and methods from existing classes. This promotes code reusability and simplifies the creation of hierarchical relationships between objects. 5. **Polymorphism:** Polymorphism allows objects of different classes to respond differently to the same message. This makes code more flexible and adaptable. 6. **Abstraction:** Abstraction refers to the user's interaction with a subset of an object's characteristics and operations.	**1M each** **(Any two correct features)**

		7. **Dynamic Binding:** In dynamic binding, the code to be executed in response to the function call is decided at runtime. Because dynamic binding is flexible, it avoids the drawbacks of static binding, which connected the function call and definition at build time. 8. **Message Passing:** Objects communicate with one another by sending and receiving information. A message for an object is a request for the execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function, and the information to be sent.	
	b)	**Define class and object.**	**2M**
	Ans.	**Class:** Class is a user defined data type that combines data and functions together. It is a collection of objects of similar type. **Object:** Object is a basic run time entity that represents a person, place or any item that the program has to handle.	**1M each for correct Definition**
	c)	**What are the rules for virtual function (write any two)**	**2M**
	Ans.	**1. Function Prototype Matching:** The prototype (including return type, name, and parameter list) of a virtual function must be identical in the base class and all derived classes that override it. This ensures that the compiler can identify the function correctly even when using a base class pointer or reference. **2. Access through Base Class Pointer or Reference:** Virtual functions are typically accessed through pointers or references of the base class type. This allows for runtime polymorphism, where the actual function called depends on the object's dynamic type at runtime, not the pointer or reference type. **3. Declaration:** Virtual functions are declared using the virtual keyword in the base class. **4. Member Function:** Virtual functions must be member functions of a class. They cannot be static members. **5. Access through Pointers or References:** Virtual functions are typically accessed through pointers or references of the base class type. This enables runtime polymorphism, where the specific function to be called is determined at runtime based on the actual object type. **6. Optional Override:** A derived class can optionally choose to override (redefine) the behavior of an inherited virtual function. This allows for specialization of the function in the derived class.	**1M each (Any 2 correct Rules)**

		7. Base Class Definition: A virtual function must be defined (have a body) in the base class, even if it's not overridden in any derived classes. **8. Friend Functions:** Virtual functions can be declared as friend functions of another class. This allows the friend function to access the virtual function directly, even if it's called through a base class pointer. **9**. **No Virtual Constructors:** Class cannot have virtual constructors, but can have virtual destructors.	
	d)	**State the use of Memory Management Operator and explain it with example.**	**2M**

	Ans.	Memory Management operators are **new** and **delete.**
		
		Uses of Memory management operators:
		
		• Dynamic memory allocation in C++ refers to performing memory allocation manually by a programmer.
		• Use of dynamically allocated memory is to allocate memory of variable size, which is not possible with compiler allocated memory except for variable-length arrays.
		• The most important use is the flexibility provided to programmers.
		• We are free to allocate and deallocate memory whenever we need it and whenever we don't need it anymore.
		
		Explanation of new and delete:
		new operator: This operator is used to allocate memory dynamically at runtime. It takes the data type of the object you want to create as an argument and returns a pointer to the newly allocated memory block.
		
		delete operator: This operator is used to deallocate memory that was previously allocated using new operator. It takes a pointer to the memory block you want to free as an argument.
		
		Example:
		
		`int* numbers = new int[10];`
		`for (int i = 0; i < 10; i++)`
		`{`
		`numbers[i] = i * i;`
		
		`}`
		
		`delete [ ] numbers;`
			1M for Any one Use & 1M for Example

	e)	**List different types of inheritance**.	**2M**
	Ans.	**Types of inheritance:** 1.Single inheritance 2. Multiple inheritance 3. Multilevel inheritance 4. Hierarchical inheritance 5. Hybrid inheritance	**½ M each (Any four types)**
	f)	**List C++ stream classes along with their function (any two classes)**	**2M**
	Ans.	**C++ stream classes:** 1. **istream class:** This class is responsible for handling input stream. It provides number of functions for handling chars, strings and objects such as get, getline, read etc. 2. **ios class:** The ios class is responsible for providing all input and output facilities to all other stream classes. 3. **ostream class:** This class is responsible for handling output stream. It provides number of functions for handling chars, strings and objects such as write, put etc. 4. **iostream class:** This class is responsible for handling both input and output stream as both istream class and ostream class is inherited into it. It provides function of both istream class and ostream class for handling chars, strings and objects such as get, getline, read, put, write etc. 5. **streambuf class:** This class provide an interface to physical devices through buffer. It acts as a base for filebuf class used for ios files.	**1M each (Any two Classes with their function)**
	g)	**Give the syntax of fclose ( ) method.**	**2M**
	Ans.	**Syntax:** int fclose(FILE* stream);	**2M for correct syntax**
2.		**Attempt any <u>THREE</u> of the following:**	**12M**
	a)	**Develop a program to find factorial of a given number using for loop.**	**4M**
	Ans.	#include<iostream> using namespace std; int main() {	**2M for correct logic &**

		int no, fact=1, i;
		
		cout<<"Enter number:";
		cin>>no;
		for(i=1;i<=no;i++)
		{
		fact=fact*i;
		}
		cout<<"Factorial ="<<fact;
		}
		Note: Any other correct logic should be considered.
			2M for correct syntax

	b)	**Explain friend function with suitable example.**	**4M**
	Ans.	**Friend function:** The private members of a class cannot be accessed from outside the class but in some situations two classes may need access of each other's private data. The common function is made friendly with all those classes whose private data need to be shared in that function. This common function is called as friend function. **Characteristics:** • Friend function is not in the scope of the class to which it has been declared as friend. • It is called without any object of class like a normal function. • It cannot access the member names directly and has to use an object name and dot membership operator with each member name. • It can be declared either in the public or the private part of a class without affecting its meaning. • It has the objects as arguments. **Example:** Program to interchange values of two integer numbers using friend function. #include<iostream> using namespace std; class B; class A {	**2M for Explanation & 2M for correct Example**

```
int x;
 public: void accept()
{
cout<<"\n Enter the value for x:";
cin>>x;
}
friend void swap(A,B);
 };
class B
{
 int y;
public: void accept()
 {
 cout<<"\n Enter the value for y:";
cin>>y;
}
 friend void swap(A,B);
 };
void swap(A a, B b)
{
cout<<"\n Before swapping:";
cout<<"\n Value for x="<<a.x;
cout<<"\n Value for y="<<b.y;
int temp;
temp =a.x;
a.x=b.y;
b.y=temp;
cout<<"\n After swapping:";
```

		cout<<"\n Value for x="<<a.x;
		cout<<"\n Value for y="<<b.y;
		}
		int main()
		{
		A a;
		B b;
		a.accept();
		b.accept();
		swap(a,b);
		}
		Note: Any other correct logic should be considered.	

	c)	**What is multilevel inheritance? Develop a C++ program for multilevel inheritance.**	**4M**
	Ans.	**Multilevel Inheritance:** The inheritance in which a class can be derived from another derived class is known as Multilevel Inheritance. Suppose there are three classes A, B, and C. A is the base class. B is the derived class of A. and C is the class that is derived from class B.	**2M for correct description & 2M for Example**

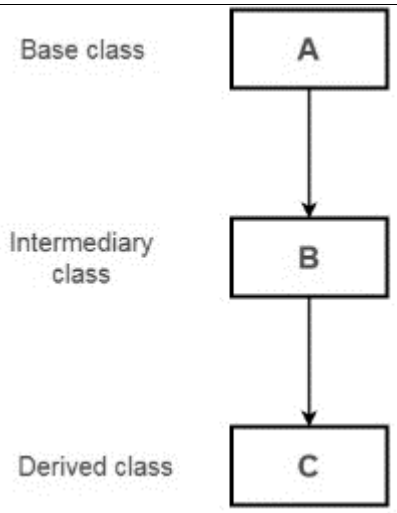


Fig: Multilevel Inheritance

Example:

```
#include<iostream>
 using namespace std;
class electronicDevice
{
   public:
      electronicDevice()
      {
         cout << "I am an electronic device.\n\n";
      }
};
class Computer: public electronicDevice
{
   public:
      Computer()
      {
         cout << "I am a computer.\n\n";
      }
};
class Linux_based : public Computer
{
   public:
      Linux_based()
      {
         cout << "I run on Linux.\n\n";;
      }
};
int main()
{
   Linux_based obj;
```

		} **Note: Any relevant example should be considered.**	
	d)	**Explain virtual function with example. Give the rules for virtual functions.**	**4M**

	Ans.	**Virtual Function:**
		
		A virtual function (also known as virtual methods) is a member function that is declared within a base class and is re-defined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the method.
		
		Example:
		
		#include<iostream>
		using namespace std;
		class base
		{
		public:
		virtual void print()
		{
		cout << "print base class\n";
		}
		
		void show()
		{
		cout << "show base class\n";
		}
		};
		
		class derived : public base
		{
		public:
		void print()
		{
		cout << "print derived class\n";
		}
		void show()
		{
		cout << "show derived class\n";
		}
		};
		int main()
		{
		base* bptr;
		derived d;
			1M Explanation, 1M for Example & 2M for Correct rules (Any four correct rules, ½ mark for each rule)

		bptr = &d;
		bptr->print();
		bptr->show();
		
		}
		Rules for virtual functions:
		1. The virtual functions must be members of some class.
		2. They cannot be static members.
		3. They are accessed by using object pointers.
		4. A virtual function can be a friend of another class.
		5. A virtual function in a base class must be defined, even though it may not be used.
		6. The prototypes of the base class version of a virtual function and all the derived class versions must be identical.
		7. We cannot have virtual constructors, but we can have virtual destructors.
		8. While a base pointer can point to any type of the derived object, the reverse is not true we cannot use a pointer to a derived class to access an object of the base type.
		9. When a base pointer points to a derived class, incrementing or decrementing it will not make it to point to the next object of the derived class it is incremented or decremented only relative to its base type.
		10. If a virtual function is defined in the base class, it need not be necessarily redefined in the derived class.	

3.		**Attempt any THREE of the following:**	**12M**
	a)	**Develop a program to declare a class student the data members are rollno, name and marks accept and display data for one object of class student.**	**4M**

	Ans.	#include<iostream>
		using namespace std;
		class student
		{
		int rollno;
		char name[20];
		float marks;
		public:
		void accept();
		void display();
		};
		void student::accept()
		{
			1M for Class Declaration, 1M for Accepting value, 1M for Displaying data & 1M for Main function object creation with correct syntax

```
cout<<"\nEnter data of student:";
cout<<"\nRoll number:";
cin>>rollno;
cout<<"\nName:";
cin>>name;
cout<<"\nMarks:";
cin>>marks;
}
void student::display()
{
cout<<"\nStudents data is:";
cout<<"\nRoll number:"<<rollno;
cout<<"\nName:"<<name;
cout<<"\nMarks:"<<marks;
}
int main()
{
student S;
 S.accept();
S.display();
}
```

b) Describe the concept of virtual base class with suitable example. **4M**

Ans. **Virtual Base Class:**

An ancestor class is declared as virtual base class which is used to avoid duplication of inherited members inside child class due to multiple path of inheritance.

2M for Description with diagram & 2M for Example

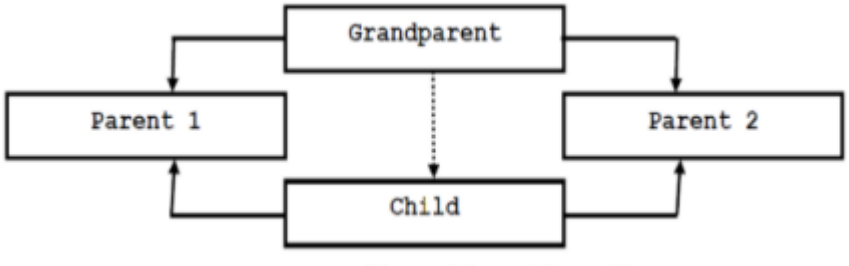


Fig.:Virtual Base Class (Multipath inheritance)

Consider a hybrid inheritance as shown in the above diagram. The child class has two direct base classes, Parent1 and Parent2 which themselves have a common base class as Grandparent. The child inherits the members of Grandparent via two separate paths. All the public and protected members of Grandparent are inherited into Child twice, first via Parent1 and again via Parent2. This leads to duplicate sets of the inherited members of Grandparent inside Child class. The duplication of inherited members can be avoided by making the common base class as virtual base class while declaring the direct or intermediate base classes as shown below.

```
class Grandparent
{
};
```

```
class Parent1: virtual public Grandparent
{
};
class Parent2: public virtual Grandparent
{
};
class Child: public Parent1, public Parent2
{
};
```

Example:

```
#include<iostream>
using namespace std;
class student
{
int rno;
public:
void getnumber()
{
cout<<"Enter Roll No:";
cin>>rno;
}
void putnumber()
{
cout<<"\n\n\t Roll No:"<<rno<<"\n";
}
};
class test: virtual public student
{
public:
int part1,part2;
void getmarks()
{
cout<<"Enter Marks\n";
cout<<"Part1:";
cin>>part1; cout<<"Part2:";
cin>>part2;
}
void putmarks()
{
cout<<"\t Marks Obtained\n";
cout<<"\n\t Part1:"<<part1;
cout<<"\n\tPart2:"<<part2;
}
};
class sports: public virtual student
{
public:
```

```
int score;
void getscore()
{
cout<<"Enter Sports Score:";
cin>>score;
}
void putscore()
{
cout<<"\n\t Sports Score is:"<<score;
}
};
class result: public test, public sports
{
int total;
public:
void display()
{
total=part1+part2+score;
putnumber();
putmarks();
putscore();
cout<<"\n\t Total Score:"<<total;
}
};
int main()
{
result obj;

obj.getnumber();
obj.getmarks();
obj.getscore();
obj.display();

}
```

Note: Any relevant example should be considered.

	c)	**Write a C++ program to declare a class college with name and code. Derive new class as student with members as name. Accept and display details of one students along with college data.**	**4M**
	Ans.	#include<c> using namespace std; class college { char name[10]; int code; public: void accept_college() { cout<<"Enter College Name:";	**1M for Definition of Base Class, 2M for Definition of Derived Class & 1M for**

		cin>>name;
		cout<<"Enter Code:";
		cin>>code;
		}
		void display_college()
		{
		cout<<endl<<"College Name:"<<name;
		cout<<endl<<"College Code:"<<code;
		}
		};
		class student:public college
		{
		char sname[10];
		public:
		void accept_student()
		{
		cout<<"Enter student Name:";
		
		cin>>sname;
		}
		void display_student()
		{
		cout<<endl<<"Student Name:"<<sname;
		}
		};
		int main()
		{
		student s;
		
		s.accept_college();
		s.accept_student();
		s.display_college();
		s.display_student();
		
		}
			Main function with correct syntax

	d)	**Write a program to count number of lines in a file.**	**4M**

	Ans.	#include<iostream>
		#include<fstream>
		using namespace std;
		int main()
		{
		int count = 0;
		string line;
		ifstream file("abc.txt");
		while (getline(file, line))
		count++;
		cout << "Numbers of lines in the file : " << count << endl;
			1M for creating and initialization of input file stream, 2M for Counting number of lines & 1M for

```
    return 0;
}
```

Note: Any other correct logic should be considered.

Printing number of lines from a file

4. **Attempt any THREE of the following:** **12M**

a) **Describe all visibility modes and effects with example.** **4M**

Ans.

Base class visibility	Derived class visibility		
	Public derivation	Private derivation	Protected derivation
Private ⟶	Not inherited	Not inherited	Not inherited
Protected ⟶	Protected	Private	Protected
Public ⟶	Public	Private	Protected

Fig.: Visibility of inherited members

Visibility modes, also known as access specifiers, are important concepts in object-oriented programming language. They control how other parts of the code can access the properties (data members) and functionalities (member functions) of a class. This is crucial for data hiding and creating well-organized code.

Visibility modes and their effects:

Public: When a base class is inherited with public visibility mode, the protected members of the base class will be inherited as protected members of the derived class and the public members of the base class will be inherited as public members of the derived class.

Private: When a base class is inherited with private visibility mode the public and protected members of the base class become 'Private' members of the derived class.

Protected: When a base class is inherited with protected visibility mode the protected and public members of the base class become 'protected members' of the derived class.

Example:

```
class A
{
public: int x;
protected: int y;
private: int z;
};
class B: public A
{
// x is public
```

3M for Description of visibility mode and its effects (1M for each mode) & 1M for Example with all 3-visibility mode

```
// y is protected
// z is not accessible from B
 };
class C: protected A
{
// x is protected
// y is protected
// z is not accessible from C
};
class D : private A
{
// x is private
// y is private
// z is not accessible from D
};
```

Note: Any relevant example should be considered.

b) Differentiate between constructor and destructor (any 4 points) — 4M

Ans. — **1M each for Any four correct differences**

Sr. No.	Constructor	Destructor
1	Constructor helps to initialize the object of a class.	Whereas destructor is used to destroy the instances.
2	It is declared as className (arguments if any) {Constructor's Body }.	Whereas it is declared as ~ className(no arguments){ }.
3	Constructor can either accept arguments or not.	While it can't have any arguments.
4	A constructor is called when an instance or object of a class is created.	It is called while object of the class is freed or deleted.
5	Constructor is used to allocate the memory to an instance or object.	While it is used to deallocate the memory of an object of a class.
6	Constructor can be overloaded.	While it can't be overloaded.
7	The constructor's name is same as the class name.	Here, its name is also same as the class name preceded by the tiled (~) operator.

8	In a class, there can be multiple constructors.	While in a class, there is always a single destructor.		
9	There is a concept of copy constructor which is used to initialize an object from another object.	While here, there is no copy destructor concept.		
10	They are often called in successive order.	They are often called in reverse order of constructor.		
11	Syntax: classname() {… … }	Syntax: destructor name is preceded with tilde. ~classname() {…. …. }		

12	Example:
	ABC()
	{
	…
	}
		Example:
		~ABC()
		{
		….
		}		

	c)	**Describe function overloading and function overriding with suitable example.**	**4M**

	Ans.	**Function Overloading:**
		
		• **Definition:** Function overloading allows to define multiple functions with the same name within the same scope but with different parameter lists. The compiler distinguishes between them based on the number, type, or order of the parameters.
		• It improves code readability and reusability.
		• We can have functions with the same name that performs different operations based on the data they receive.
		
		Example:
		#include <iostream>
		using namespace std;
		// overloaded functions
		void test(int);
		void test(float);
		void test(int, float);
			1M for description & 1M for Example of each concept

```
int main()
{
   int a = 5;
   float b = 5.5;

   // Overloaded functions with different type and number of parameters
   test(a);
   test(b);
   test(a, b);

   return 0;
}

// Method 1
void test(int var)
{
   cout << "Integer number: " << var << endl;
}

// Method 2
void test(float var)
{
   cout << "Float number: "<< var << endl;
}

// Method 3
void test(int var1, float var2)
{
   cout << "Integer number: " << var1;
   cout << " and float number:" << var2;
}
```

Function Overriding:

- **Definition:** Function overriding occurs in inheritance. When a derived class inherits from a base class and redefines a member function of the base class with the same name and same parameter (return type and parameter list), it's called overriding.
- The derived class's function implementation overrides the base class's function for the object of the derived class.

Example:

```
#include<iostream>
using namespace std;
class BaseClass
{
public:
   virtual void Display()
```

```
        {
            cout << "\nThis is Display() method of BaseClass";
        }
        void Show()
        {
            cout << "\nThis is Show() method of BaseClass";
        }
    };

    class DerivedClass : public BaseClass
    {
    public:
        // Overriding method - new working of
        // base class display method
        void Display()
        {
            cout << "\nThis is Display() method of DerivedClass";
        }
    };

    // Driver code
    int main()
    {
        DerivedClass dr;
        BaseClass &bs = dr;
        bs.Display();
        dr.Show();
}
```

Note: Any relevant example should be considered.

	d)	**Develop a C++ program to read content of file abc.txt**	**4M**
	Ans.		**1M for creating and initialization of input file stream, 2M for Reading the content of file & 1M for Printing each line to the console of a file**

```
#include <iostream>
#include <fstream>
using namespace std;
int main() {
  ifstream inputFile("abc.txt");

  // Check if the file was opened successfully
  if (inputFile.is_open()) {
    string line;

    // Read the file line by line
    while (getline(inputFile, line)) {
      // Print each line to the console
      cout << line << endl;
    }

    inputFile.close();
  } else {
```

```
    //cerr << "Error: Could not open file 'abc.txt'." << endl;
    cout << "Error:'abc.txt' File not found." << endl;
  }

  return 0;
}
```

e) With suitable example describe structure of C++ program. 4M

Ans. General C++ program has following structure: **2M for Description with diagram & 2M for Example**

Include files
Class declaration
Member functions definitions
Main function program

Fig.: Structure of a C++ program

Description:

1. Include files
In this section a programmer includes all header files which are require to execute given program. The most important file is iostream.h header file. This file defines most of the C++statements like cout and cin. Without this file one cannot load C++ program.

2. Class Declaration
In this section a programmer declares all classes which are necessary for given program. The programmer uses general syntax of creating class.

3. Member Functions Definitions
This section allows programmer to design member functions of a class. The programmer can have inside declaration of a function or outside declaration of a function.

4. Main Function Program

In this section programmer creates objects and calls various functions writer within various class.

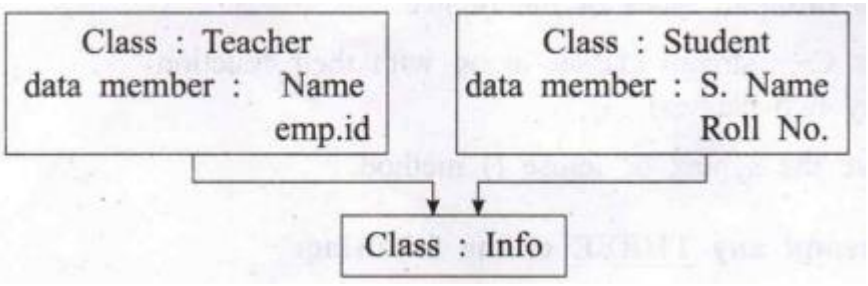
Example:

```
#include<iostream>
using namespace std;
```

```
class MyClass        // The class
{
 public:           // Access specifier
   int myNum;       // Attribute (int variable)
   string myString;  // Attribute (string variable)
};
int main()
{
  MyClass myObj;  // Create an object of MyClass

  // Access attributes and set values
  myObj.myNum = 15;
  myObj.myString = "Some text";

  // Print attribute values
  cout << myObj.myNum << "\n";
  cout << myObj.myString;
  return 0;
}
```

5.		**Attempt any TWO of following.**	**12M**
	a)	**Write a C++ program to implement inheritance as shown in fig:** 	**6M**

	Ans.	#include <iostream>
		using namespace std;
		// Base class Teacher
		class Teacher {
		public:
		string Name;
		int emp_id;
		
		void setTeacherDetails()
		{
		cout<<"Enter Employee Name:";
		cin>>Name;
		
		cout<<"Enter Employee ID:";
		cin>>emp_id;
		}
		
		void displayTeacherDetails()
			2M for class Definition, 2M for Correct implementation & 2M for logic

```
{
      cout << "Employee Name: " << Name << endl;
      cout << "Employee ID: " << emp_id << endl;
   }
};
// Base class Student
class Student {
public:
   string S_Name;
   int Roll_No;

   void setStudentDetails()
  {
      cout<<"Enter Student Name:";
      cin>>S_Name;

     cout<<"Enter Roll Number ID:";
     cin>>Roll_No;
   }
   void displayStudentDetails() {
      cout << "Student Name: " << S_Name << endl;
      cout << "Roll Number: " << Roll_No << endl;
   }
};

// Derived class Info inheriting from both Teacher and Student
class Info : public Teacher, public Student {
public:
   void displayInfo() {
      displayTeacherDetails();
      displayStudentDetails();
   }
};

int main() {
   Info info;
   info.setTeacherDetails();
   info.setStudentDetails();
   cout<<endl;
   info.displayInfo();
}
```

OR

```
#include <iostream>
using namespace std;
// Base class Teacher
class Teacher {
public:
```

```
    string Name;
    int emp_id;

    void setTeacherDetails(string name, int id) {
        Name = name;
        emp_id = id;
    }

    void displayTeacherDetails() {
        cout << "Teacher Name: " << Name << endl;
        cout << "Employee ID: " << emp_id << endl;
    }
};

// Base class Student
class Student {
public:
    string S_Name;
    int Roll_No;

    void setStudentDetails(string name, int rollNo) {
        S_Name = name;
        Roll_No = rollNo;
    }

    void displayStudentDetails() {
        cout << "Student Name: " << S_Name << endl;
        cout << "Roll Number: " << Roll_No << endl;
    }
};

// Derived class Info inheriting from both Teacher and Student
class Info : public Teacher, public Student {
public:
    void displayInfo() {
        displayTeacherDetails();
        displayStudentDetails();
    }
};
int main() {
    Info info;

    info.setTeacherDetails("Alice", 123);
    info.setStudentDetails("Bob", 456);

    info.displayInfo();

}
```

	b)	**Develop a program to declare class book containing data members as title ,authorname ,publication, price accept and display the information for one object using pointer to that objects.**	**6M**

	Ans.	#include<iostream>
		using namespace std;
		class book
		{
		char title[30];
		char authorname[30];
		char publication[30];
		int price;
		
		public:
		void accept()
		{
		cout<<"\n Enter the title of Book:";
		cin>>title;
		cout<<"\n Enter the Book Author Name:";
		cin>>authorname;
		cout<<"\n Enter the Publication details:";
		cin>>publication;
		cout<<"\n Enter the Price of Book:";
		cin>>price;
		}
		void display()
		{
		cout<<"\n Title of the Book:"<<title;
		cout<<"\n The Name of Author is:"<<authorname;
		cout<<"\n The Publication company is:"<<publication;
		cout<<"\n The Price of the Book is:"<<price;
		}
		};
		
		int main()
		{
		book b,*p;
		p=&b;
		
		p->accept();
		p->display();
		return 0;
		}
			2M for class definition 1M for Accepting Data, 1M for Displaying Data & 2M for pointer to object declaration & implementation

	c)	**Develop a C++ program to add two 3x3 matrices and display addition.**	**6M**
	Ans.	#include<iostream> using namespace std; int main()	**2M for input of Two Matrices,**

```
{
        int a[3][3],b[3][3],c[3][3];
        int i,j;

        cout<<"\n Enter the matrix for A:";
        for(i=0;i<3;i++)
        {
                for(j=0;j<3;j++)
                {
                        cin>>a[i][j];
                }
        }

        cout<<"\n Enter the matrix for B:";
        for(i=0;i<3;i++)
        {
                for(j=0;j<3;j++)
                {
                        cin>>b[i][j];
                }

        }

        for(i=0;i<3;i++)
        {
                for(j=0;j<3;j++)
                {
                    c[i][j]=a[i][j]+b[i][j];
                }
        }

        cout<<"\n The addition of two matrices A and B is:";
        for(i=0;i<3;i++)
        {
                cout<<"\n";

                for(j=0;j<3;j++)
                {
                    cout<<"\t"<<c[i][j];
                }
        }

}
```

2M for Addition of Two Matrices & 2M for Displaying Addition of Matrix

6.		**Attempt any TWO of following.**	**12M**
	a)	**Develop a C++ program to create structure book with data members name, cost and author. Accept and display data for 5 books using structure**	**6M**

	Ans.	#include<iostream>
		using namespace std;
		struct book
		{
		char name[30];
		int cost;
		char author[30];
		}b[5];
		
		int main()
		{
		int i;
		
		for(i=0;i<5;i++)
		{
		cout<<"\n Enter the Book "<<i+1<<" Name:";
		cin>>b[i].name;
		
		cout<<"\n Enter the Book "<<i+1<<" Cost:";
		cin>>b[i].cost;
		
		cout<<"\n Enter the Author Name for Book "<<i+1<<":";
		cin>>b[i].author;
		}
		
		cout<<"\n Book Information:"<<endl;
		
		for(i=0;i<5;i++)
		{
		cout<<"\n The Book "<<i+1<<" Name:"<<b[i].name;
		
		cout<<"\n The Book "<<i+1<<" Cost:"<<b[i].cost;
		
		cout<<"\n The Author Name for Book "<<i+1<<":"<<b[i].author;
		
		cout<<endl;
		}
		}
			2M for correct Code for Structure Definition, 2M for Accepting data & 2M for Displaying data

	b)	**Develop a C++ program using parameterized constructor**	**6M**
	Ans.	#include<iostream> using namespace std; class number { int x; public:	**2M for class Definition, 2M for constructor Definition**

```
        number(int y)
        {
            x=y;
        }
        void display( )
        {
            cout<<"The sqaure of number is:"<<x*x;
        }
    };
    int main( )
    {
        number n(50);

        n.display( );
    }
```

Note: Any other relevant program should be considered.

& 2M for Correct implementation

c) Write a C++ program to overload "+" operator so that it will perform concatenation of two strings. (Use class get data function to accept two strings.) — **6M**

Ans.

```
#include<iostream>
#include<string.h>

using namespace std;
class String
{
  char str[20]; //member variable for string input
public:
  void input( ) //member function
  {
      cout<<"Enter String: ";
      cin>>str;
  }
  void display( ) //member function for output
  {
      cout<<"Concatenation of two Strings: "<<str;
  }
  String operator+(String s) //operator overloading
  {
      String obj;
      strcat(str,s.str);
      strcpy(obj.str,str);
      return obj;
  }
};
int main( )
{
  String str1,str2,str3; //creating three object
  str1.input( );
```

2M for class Definition, 2M for Operator function, & 2M for correct implementation

```
   str2.input( );
   str3=str1+str2;
   str3.display( ); //displaying

}
```

OR

```
#include<iostream>
#include<string.h>
using namespace std;
        class string_concat
        {
                char s1[30];
                char s2[30];

                public:
                void getdata( )
                {
                        cout<<"\n Enter the First String :";
                        cin>>s1;

                        cout<<"\n Enter the Second String :";
                        cin>>s2;

                }
                void  operator+( )
                {
                        cout<<"\n The concatenation of two String
is:"<<strcat(s1,s2);
                }
        };
                int main( )
                {
                        string_concat s;
                        s.getdata( );
                        +s;
                }
```